AIORI-3 HACKATHON

# CONNECTIQ

Explainable Home and Last-Mile Connectivity Diagnosis

---

**Problem Statement:** PS-015

**Team:** CONNECTIQ

**Project Track:** Smart Cities

Technical Project Report

Prepared for AIORI-3 project submission

Implementation status and test results are reported as observed. Features listed under limitations are not claimed as completed.

# 1. Project Overview

CONNECTIQ is a desktop-based network diagnostics prototype designed to help a user examine common home and last-mile connectivity problems. The prototype collects basic network measurements, applies transparent rule-based checks, and presents findings with supporting evidence and suggested troubleshooting actions.

The project follows the measurement-oriented approach described in the problem statement. It focuses on reachability, DNS response, TCP connection timing, and ICMP packet loss. The current prototype is an initial implementation and does not yet cover every requested measurement or fault-localization scenario.

# 2. Problem Statement

Home and last-mile connectivity issues are difficult for users to diagnose because symptoms such as slow browsing, failed websites, and unstable connections can arise from different network layers. Users often lack clear evidence to distinguish name-resolution problems, target reachability failures, packet loss, and other causes.

PS-015 asks for an explainable diagnostic workflow that gathers network measurements, identifies likely fault conditions, and produces an understandable report. When the available evidence is insufficient, the system should return an inconclusive result rather than making an unsupported diagnosis.

# 3. Objectives

- Check whether a selected remote host is reachable over TCP.
- Measure DNS lookup response time.
- Collect repeated TCP connection-time samples.
- Estimate ICMP packet loss using system ping.
- Apply transparent, rule-based diagnostic logic.
- Display findings, evidence, recommendations, and limitations in a Streamlit dashboard.
- Generate a downloadable JSON diagnostic report.

# 4. Proposed System Architecture

The implemented workflow is organized into the following stages:

| Stage | Responsibility |
|---|---|
| User Interface | Streamlit dashboard accepts a target hostname and displays the diagnostic result. |
| Probe Execution | Runs reachability, DNS, repeated TCP timing, and ICMP packet-loss checks. |
| Telemetry Collection | Collects status, timing values, errors, and probe outputs. |
| Rule-Based Diagnosis | Evaluates collected evidence and assigns a diagnostic status with findings. |
| Report Generation | Creates a structured JSON report for saving or sharing. |

Data flow: User Input → Probe Scheduler / Probe Functions → Telemetry → Rule-Based Diagnosis → Dashboard and JSON Report.

# 5. Tools and Technologies

- **Programming language:** Python.
- **Dashboard:** Streamlit.

- **Network measurements:** Python socket and DNS-resolution functions, plus the operating system ping command for ICMP testing.
- **Testing:** pytest and manually constructed diagnostic scenarios.
- **Development environment:** Visual Studio Code and PowerShell on Windows.
- **Version control:** Git and GitHub.

# 6. Implementation Details

## 6.1 Reachability check

The prototype attempts a TCP connection to the selected hostname and port. A successful connection is recorded as reachable. A failed connection is recorded with the available error information. TCP reachability does not prove that every internet service or network path is healthy.

## 6.2 DNS response measurement

The DNS probe resolves the selected hostname and records the lookup duration and returned address information when available. A lookup failure is retained as evidence for the diagnosis engine.

## 6.3 Repeated TCP timing

The prototype performs repeated TCP connection attempts and records successful samples, average, minimum, maximum, and success count. These values represent TCP connection setup timing. They are not pure ICMP round-trip time measurements.

## 6.4 ICMP packet-loss measurement

The prototype invokes the system ping utility and parses its result to estimate packet loss for the selected target. ICMP traffic may be filtered or rate-limited by a target or network, so packet loss in this probe does not always indicate a general internet connection fault.

## 6.5 Explainable diagnosis

The diagnosis engine uses explicit rules and the collected probe results. It supports three outcome categories:

- **NO_ISSUES_DETECTED:** the available checks did not identify a fault.
- **ISSUES_DETECTED:** one or more checks produced evidence of a problem.
- **INCONCLUSIVE:** the available measurements did not provide enough evidence for a reliable conclusion.

The engine avoids treating missing or unknown reachability data as a confirmed failure. Each reported finding includes evidence and a suggested next action.

# 7. Dashboard and Report Output

The Streamlit dashboard presents the connectivity result and the probe measurements. It also displays the diagnosis status, findings, evidence, recommendations, and known limitations. The application generates a JSON report that preserves the diagnostic output for later review.

To run the dashboard from the project root in the VS Code PowerShell terminal, activate the project virtual environment if required, then run:

```
python -m streamlit run app.py
```

Open the local address printed by Streamlit, normally http://localhost:8501.

# 8. Testing and Results

The project was checked using automated tests and manually constructed diagnostic inputs. The latest recorded automated test run completed with 2 tests passed.

| Test scenario | Expected / observed behavior |
|---|---|
| Healthy probe data | NO_ISSUES_DETECTED; evidence from the available probes was counted. |
| DNS resolution failure | ISSUES_DETECTED with a DNS-related finding. |
| ICMP packet loss scenario | ISSUES_DETECTED with a packet-loss finding. |
| Repeated TCP failures | ISSUES_DETECTED with reachability / TCP failure findings. |
| Unknown or missing probe data | INCONCLUSIVE; unknown status is not reported as a confirmed fault. |

A previously recorded live probe run against google.com returned successful DNS resolution, 5/5 TCP connection attempts, and 0.0% ICMP packet loss. Timing values vary by network conditions and test time, so this sample is illustrative rather than a benchmark.

## 9. Current Limitations

- Loaded jitter measurement is not implemented.
- Gateway-level fault diagnosis is not implemented.
- Controlled network congestion and packet-loss fault injection are not implemented as an integrated test environment.
- Scheduled measurements across multiple volunteer or test endpoints are not implemented.
- TCP connection timing is not equivalent to pure round-trip latency.
- ICMP results depend on target and network filtering behavior.
- The current rules provide basic evidence-based guidance, not a complete localization of every home or last-mile fault.

## 10. Future Enhancements

- Add scheduled probes and configurable multiple endpoints.
- Measure unloaded and loaded latency variation and jitter using an appropriate method.
- Add gateway reachability checks and local network path evidence.
- Add controlled fault scenarios for DNS failure, packet loss, and congestion.
- Improve the inference rules while retaining transparent evidence for each conclusion.
- Add report export formats and a hosted demonstration deployment.

## 11. Conclusion

CONNECTIQ delivers an initial explainable network diagnostics prototype. It collects basic reachability, DNS, TCP timing, and ICMP packet-loss evidence, then presents rule-based findings through a Streamlit dashboard and JSON report. Automated and scenario-based checks verify the core diagnostic categories, including an inconclusive outcome when evidence is insufficient.

The prototype remains under development. The limitations in this report identify the major areas required to move toward the full PS-015 objective. The team should demonstrate the working dashboard and describe completed features separately from planned enhancements.

## 12. Repository and Submission Details

GitHub repository: https://github.com/aiori-hackathon/A3-PS015-TC241

Presentation file in repository: CONNECTIQ-AIORI3-PS015-Presentation.pdf

Main application entry point: app.py

Core implementation modules: src/diagnostics.py and src/network_probes.py

Automated tests: tests/

---

Prepared by Team CONNECTIQ | AIORI-3 | PS-015